

Implementing RESTful API with JPA

With Java 25 and Spring Boot 4

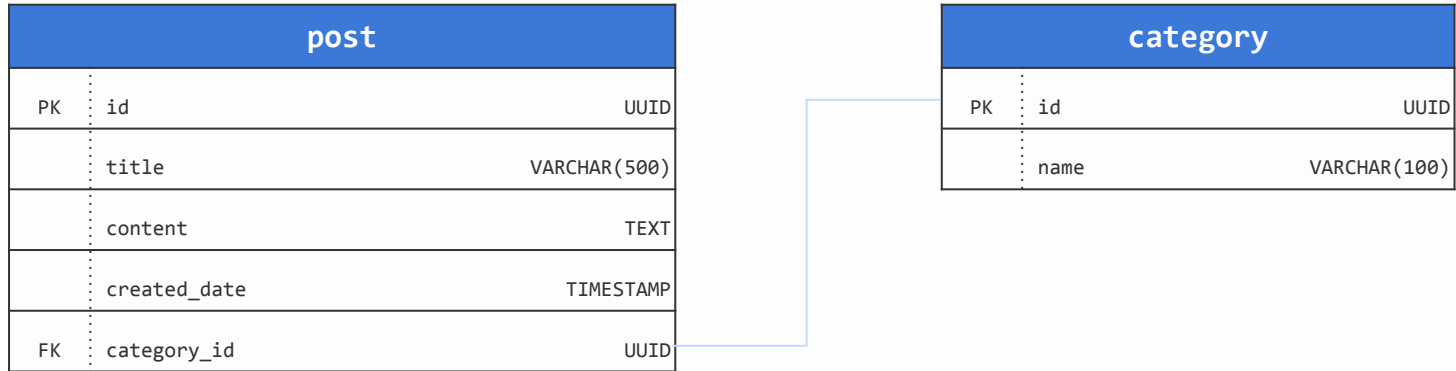
Session 03

- Create a spring boot application
blogger-box-backend
- Publish project on **Github** Repository
- Expose our first **endpoints**
- Document your endpoints with **Swagger**
- HTTP request **methods** with **conventions**
- Expose all **endpoints** for a blogger platform

Session 04

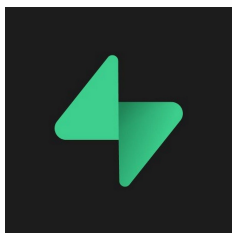
- Create a **database** remotely
- Connect backend to a **database** via **JPA**
- **Http code**
- **Exception** handling

Database diagram of a blogging platform

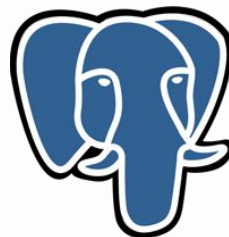


Remote PostgreSQL database

Create and **Connect** to a remotely **PostgreSQL** database via **Supabase**



Supabase



PostgreSQL



Remote PostgreSQL DB

Head over to supabase.com to create your PostgreSQL database remotely

1. Sign up
2. Create a new organization
3. Create a new project

Project name	blogger-dauphine
Database password	< generate a password >
Region	West EU (Paris)



> Organization: devt > Create a new project > Extend your database



Create a new project

Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization

devt

Project name

blogger-dauphine

Database Password

.....

Copy

This password is strong. [Generate a password](#)

Region

 West EU (Paris)

Select the region closest to your users for the best performance.

SECURITY OPTIONS >

ADVANCED CONFIGURATION >

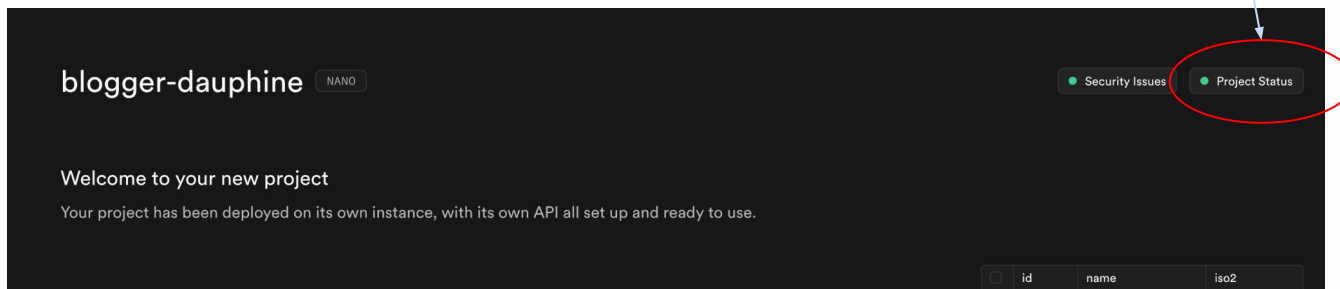
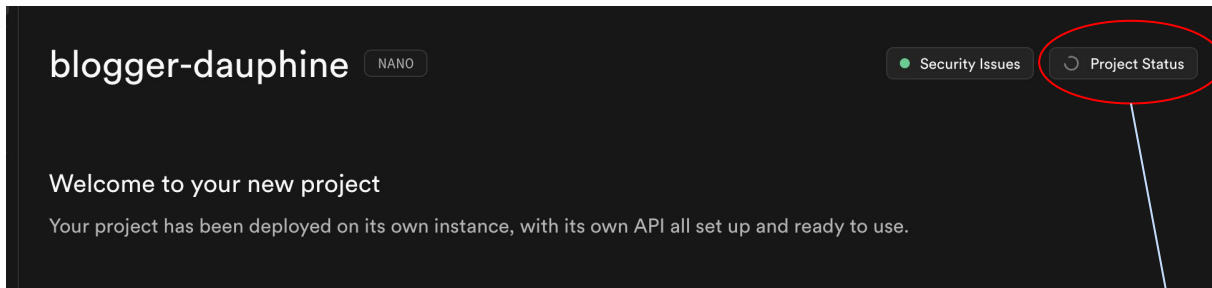
Cancel

You can rename your project later

Create new project

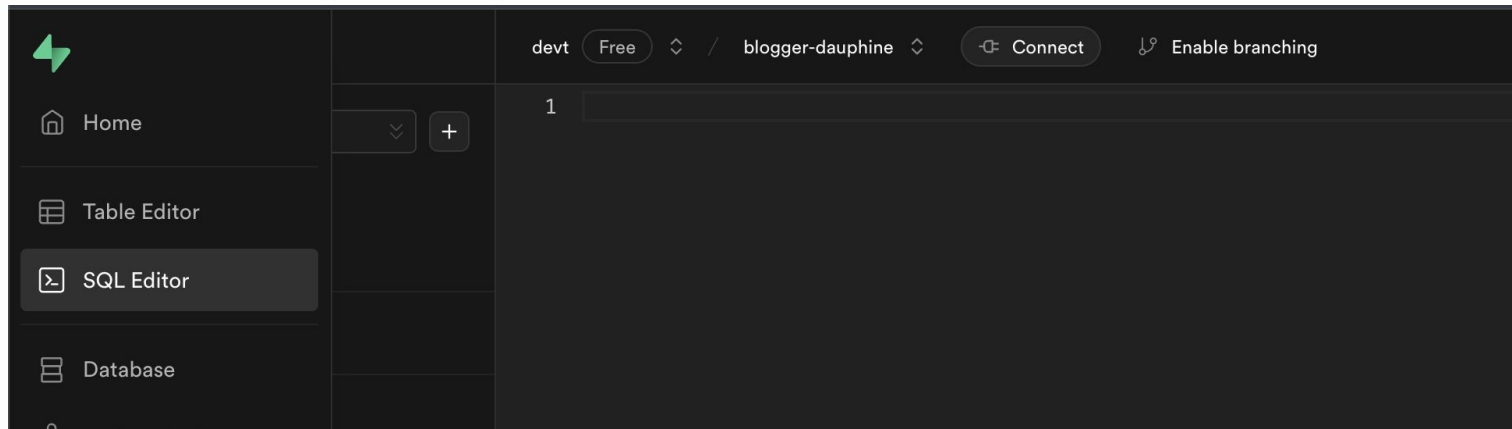


Wait until the Database is ready





Populate blogger database



Use Supabase SQL Editor to create category and post tables



Category Model

```
Category.java x
1 package com.dauphine.blogger.models;
2
3 import java.util.UUID;
4
5 public class Category { 12 usages
6
7     private UUID id; 2 usages
8     private String name; 2 usages
9
10    public UUID getId() { 3 usages
11        return id;
12    }
13
14    public void setId(UUID id) { no usages
15        this.id = id;
16    }
17
18    public String getName() { no usages
19        return name;
20    }
21
22    public void setName(String name) { 1 usage
23        this.name = name;
24    }
25 }
```

category		
PK	id	UUID
	name	VARCHAR(100)



Post model

post		
PK	id	UUID
	title	VARCHAR(100)
	content	TEXT
	created_date	TIMESTAMP
FK	category_id	UUID

category		
PK	id	UUID
	name	VARCHAR(100)

```
Post.java
1 package com.dauphine.blogger.models;
2
3 import java.time.LocalDateTime;
4 import java.util.UUID;
5
6 public class Post {
7     private UUID id;
8     private String title;
9     private String content;
10    private LocalDateTime createdDate;
11    private Category category;
12
13    public UUID getId() {
14        return id;
15    }
16
17    public void setId(UUID id) {
18        this.id = id;
19    }
20
21    public String getTitle() {
22        return title;
23    }
24
25    public void setTitle(String title) {
26        this.title = title;
27    }
28
29    public String getContent() {
30        return content;
31    }
32
33    public void setContent(String content) {
34        this.content = content;
35    }
36
37    public LocalDateTime getCreatedDate() {
38        return createdDate;
39    }
40
41    public void setCreatedDate(LocalDateTime createdDate) {
42        this.createdDate = createdDate;
43    }
44
45    public Category getCategory() {
46        return category;
47    }
48
49    public void setCategory(Category category) {
50        this.category = category;
51    }
52 }
53
```



Populate database

Create tables (Category and Post)

Add some data into the created table

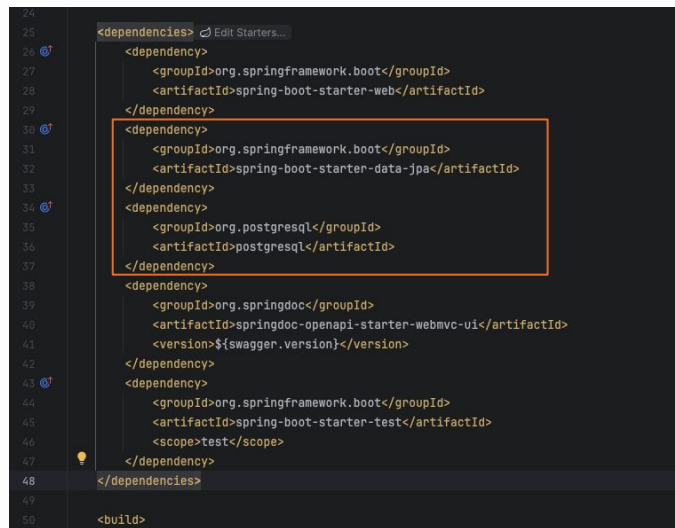


check slide 45 from session 01...

Connect to the database

In order to connect to the database, let's add a new dependency Data JPA and Postgres , which will allow to persist into a database

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```



```
25 <dependencies>
26   <dependency>
27     <groupId>org.springframework.boot</groupId>
28     <artifactId>spring-boot-starter-web</artifactId>
29   </dependency>
30   <dependency>
31     <groupId>org.springframework.boot</groupId>
32     <artifactId>spring-boot-starter-data-jpa</artifactId>
33   </dependency>
34   <dependency>
35     <groupId>org.postgresql</groupId>
36     <artifactId>postgresql</artifactId>
37   </dependency>
38   <dependency>
39     <groupId>org.springdoc</groupId>
40     <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
41     <version>${swagger.version}</version>
42   </dependency>
43   <dependency>
44     <groupId>org.springframework.boot</groupId>
45     <artifactId>spring-boot-starter-test</artifactId>
46     <scope>test</scope>
47   </dependency>
48 </dependencies>
49
50 <build>
```

Configure database in properties

```
application.properties x
1 spring.application.name=blogger-box-backend
2
3 server.port=1234
4
5 spring.datasource.url=jdbc:postgresql://REPLACE_WITH_DB_HOST:REPLACE_ME_WITH_DB_PORT/REPLACE_ME_WITH_DB_NAME
6 spring.datasource.username=REPLACE_ME_WITH_DB_USERNAME
7 spring.datasource.password=REPLACE_ME_WITH_DB_PASSWORD
8 spring.datasource.driver-class-name=org.postgresql.Driver
9 spring.jpa.hibernate.ddl-auto=validate
```

Database
connections

Deactivate JPA hibernate auto generation

```
spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=validate
```

Configure database in properties

```
spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}  
spring.datasource.username=${DB_USERNAME}  
spring.datasource.password=${DB_PASSWORD}  
  
spring.jpa.hibernate.ddl-auto=validate
```



Do not use university wifi, Supabase connection might be blocked

Change our model classes into entities

```
Category.java x
1 package com.dauphine.blogger.models;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.Id;
6 import jakarta.persistence.Table;
7
8 import java.util.UUID;
9
10 @Entity 25 usages
11 @Table(name = "category")
12 public class Category {
13
14     @Id
15     @Column(name = "id")
16     private UUID id;
17
18     @Column(name = "name") 3 usages
19     private String name;
20
21     public Category(UUID id, String name) { 4 usages
22         this.id = id;
23         this.name = name;
24     }
25
26     public Category() {
27     }
28
29     public UUID getId() { return id; }
30
31     public void setId(UUID id) { this.id = id; }
32
33     public String getName() { return name; }
34
35     public void setName(String name) { this.name = name; }
36 }
37
38
39
40
41
42
43
44
45
```

Table and column map

Very important to have an **empty constructor** (add one in case you have created a new one that takes parameter) and **getters** and **setters** for each property

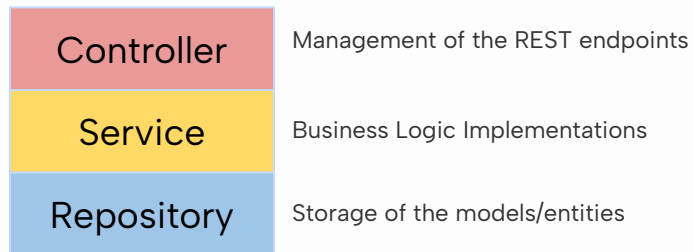
Change our model classes into entities

```
13 @Entity 28 usages
14 @Table(name = "post")
15 public class Post {
16
17     @Id
18     @Column(name = "id")
19     private UUID id;
20
21     @Column(name = "title") 3 usages
22     private String title;
23
24     @Column(name = "content") 3 usages
25     private String content;
26
27     @Column(name = "created_date") 3 usages
28     private LocalDateTime createDate;
29
30     @ManyToOne 3 usages
31     @JoinColumn(name = "category_id")
32     private Category category;
33
34     public Post() {
35     }
36
37     public Post(String title, no usages
38                 String content,
39                 Category category) {
40         this.id = UUID.randomUUID();
41         this.title = title;
42         this.content = content;
43         this.createDate = LocalDateTime.now();
44         this.category = category;
45     }
}
```

Table &
Column
mapping

Many to one relation in Post, as a post have only 1 category but the categories can have many posts, hence the ManyToOne annotation

Repository layer



```
CategoryRepository.java x
1 package com.dauphine.blogger.repositories;
2
3 import com.dauphine.blogger.models.Category;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.UUID;
7
8 public interface CategoryRepository extends JpaRepository<Category, UUID> {
9 }
10
```



JPA repository allows to generate data storage operation (SELECT, UPDATE, DELETE, ...) queries without the need to write any ...

Entity
model
class

Primary
key type

Inject Repository in service

```
10
11 @Service
12 public class CategoryServiceImpl implements CategoryService {
13
14     private final CategoryRepository repository; 6 usages
15
16     public CategoryServiceImpl(CategoryRepository repository) {
17         this.repository = repository;
18     }
19
```

Retrieve all existing categories

We will use `findAll` method already implemented in `JpaRepository`

```
20      @Override 1 usage
21      public List<Category> getAll() {
22          return repository.findAll();
23      }
```

Retrieve an existing category by id

We will use findById method already implemented in JpaRepository

```
25      @Override 2 usages
26      public Category getById(UUID id) {
27          return repository.findById(id)
28              .orElse( other: null);
29      }
```

Create new category

We will use save method already implemented in JpaRepository which will create the row in database if it doesn't exists

```
31      @Override
32      public Category create(String name) {
33          Category category = new Category(name);
34          return repository.save(category);
35      }
```

Update

We will use save method already implemented in JpaRepository which will update the row in database if it does exists

```
37      @Override 1 usage
38      public Category updateName(UUID id, String name) {
39          Category category = getById(id);
40          if (category == null) {
41              return null;
42          }
43          category.setName(name);
44          return repository.save(category);
45      }
```

Delete

We will use `deleteById` method already implemented in `JpaRepository`

```
47      @Override 1 usage
48      public boolean deleteById(UUID id) {
49          repository.deleteById(id);
50          return true;
51      }
52  }
```

Repository

If you want to see the queries being executed on each repository method, you can enable showing sql queries in properties

```
spring.jpa.show-sql=true
```

```
2025-03-29T14:03:14.483+01:00 INFO 6660 --- [blogger-box-backend] [nio-1234-exec-1] o.s.web.servlet.DispatcherServlet  
Hibernate: select p1_0.id,p1_0.category_id,p1_0.content,p1_0.created_date,p1_0.title from post p1_0  
Hibernate: select c1_0.id,c1_0.name from category c1_0 where c1_0.id=?  
Hibernate: select c1_0.id,c1_0.name from category c1_0 where c1_0.id=?  
Hibernate: select c1_0.id,c1_0.name from category c1_0 where c1_0.id=?
```



Sync with Github

add `category` repository layer with JPA

Add repository layer for Post

Add repository layer for Post and complete service layer



Sync with Github

add `post` repository layer with JPA

Formatting response

Each post have a created date

```
{
  "id": "d3e1ab20-5730-4e4a-9dc5-dac4b6f1403a",
  "title": "The Transformative Power of Foster Care Adoption",
  "content": "Foster care adoption provides permanent homes, stability and love. This article highlights the stories shared on both the children and their new families.",
  "category": {
    "id": "b04d4d87-7549-459b-a17d-b21f73da04cd",
    "name": "Adoption"
  },
  "createdDate": "2025-02-11T15:17:29.383842"
},
```

Formatting response

Setting format of date to `yyyy-MM-dd'T'HH:mm:ss.SSS'Z'`

```
@Column(name = "created_date")  
@JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss.SSS'Z'")  
private LocalDateTime createDate;
```

Formatting response

Each post have a created date

```
{
  "id": "d3e1ab20-5730-4e4a-9dc5-dac4b6f1403a",
  "title": "The Transformative Power of Foster Care Adoption: Chan
  "content": "Foster care adoption provides permanent homes for ch
t stability and love. This article highlights the stories of familie
had on both the children and their new families.",
  "category": {
    "id": "b04d4d87-7549-459b-a17d-b21f73da04cd",
    "name": "Adoption"
  },
  "createdDate": "11-02-2025 15:17:29"
},
```



Sync with Github

format category date in post

Exposing a new endpoint that finds all categories by name

Get all categories

```
GET /v1/categories
```

Get all categories by name

```
GET /v1/categories?name=Child
```

Exposing a new endpoint that finds all categories by name

We can write custom queries the repository layer, by default, the query definition uses JPQL.

Here an example on how to find all categories by name

```
public interface CategoryRepository extends JpaRepository<Category, UUID> {  
  
    @Query(""" 1 usage  👤 Elie Daher  
        SELECT category  
        FROM Category category  
        WHERE UPPER(category.name) LIKE UPPER(CONCAT('%', :name , '%'))  
        """)  
    List<Category> findAllLikeName(@Param("name") String name);  
  
}
```


Exposing a new endpoint that find all categories by name

And we will modify the following endpoint

```
@GetMapping  Elie Daher
@Operation(
    summary = "Get all categories",
    description = "Retrieve all categories or filter like name"
)
public List<Category> getAll(@RequestParam(required = false) String name) {
    List<Category> categories = name == null || name.isBlank()
        ? categoryService.getAll()
        : categoryService.getAllLikeName(name);
    return categories;
}
```

Get all categories

GET /v1/categories

Get all categories by name

GET /v1/categories?name=Child

Expose an endpoint that find all posts by title or content

Get all posts by title or content

```
GET /v1/posts?value=...
```



Sync with Github

exposing ``post`` / ``category`` filtering endpoints

HTTP status code

HTTP defines these standard status codes that can be used to convey the results of a client's requests. They are divided into five categories

1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client error
5xx	Server error

Common http status code

Request was successful

200 OK

Request was successful and a new resource was created

201 Created

Server couldn't process the request to a client error

400 Bad request

Server encountered an unexpected exception

500 Internal server error

User don't have the permissions to access the resource

403 Forbidden

Resource not found

404 Not Found

Server can not handle requests

503 Service unavailable

Example for OK (200) use case

@GetMapping  Elie Daher

@Operation(
summary = "Get all categories",
description = "Retrieve all categories or filter like name"
)

```
public ResponseEntity<List<Category>> getAll(@RequestParam(required = false) String name) {  
    List<Category> categories = name == null || name.isBlank()  
        ? categoryService.getAll()  
        : categoryService.getAllLikeName(name);  
    return ResponseEntity.ok(categories);  
}
```

Code

Details

200

Response body

```
[  
  {  
    "id": "fe801430-9a66-4dd9-8c67-9b4793d09ec0",  
    "name": "Adoption"  
  },  
  {  
    "id": "e576cae7-a242-4b6f-b8b3-dc0548d00656",  
    "name": "Children"  
  },  
  {  
    "id": "5399f229-17aa-40d5-8239-730927f95464",  
    "name": "Animals"  
  }  
]
```

Example for Created (201) use case

```
@PostMapping @Elie Daher
@Operation(
    summary = "Create new category",
    description = "Create new category, only required field is the name of the category to create"
)
public ResponseEntity<Category> create(@RequestBody CategoryRequest categoryRequest) throws CategoryNameAlreadyExistsException {
    Category category = categoryService.create(categoryRequest.getName());
    return ResponseEntity
        .created(URI.create("v1/categories/" + category.getId()))
        .body(category);
}
```

201

Undocumented

Response body

```
{
  "id": "4a90980b-cc5b-46a9-ade6-c3b2cd52a861",
  "name": "My category"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed, 24 Apr 2024 14:08:02 GMT
keep-alive: timeout=60
location: v1/categories/4a90980b-cc5b-46a9-ade6-c3b2cd52a861
transfer-encoding: chunked
```

Example for Not found (404) use case

```
@GetMapping("/{id}")  Elie Daher *
@Operation(
    summary = "Get category by id",
    description = "Retrieve a category by id"
)
public ResponseEntity<Category> getById(@PathVariable UUID id) {
    Category category = categoryService.getById(id);
    if (category == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(category);
}
```

Code

Details

404

Undocumented

Error: response status is 404

Response headers

```
connection: keep-alive
content-length: 0
date: Wed, 24 Apr 2024 14:24:12 GMT
keep-alive: timeout=60
```


Example for Not found (404) use case

We can enhance it, by returning an exception in the service layer instead of `null`

```
@Override 2 usages  
public Category getById(UUID id) {  
    return repository.findById(id)  
        .orElse( other: null);  
}
```



```
@Override 4 usages Elie Daher  
public Category getById(UUID id) throws CategoryNotFoundException {  
    return categoryRepository.findById(id)  
        .orElseThrow(() -> new CategoryNotFoundException(id));  
}
```

Example for Not found (404) use case

And that way in the controller we will catch this exception and throw the adequate response

```
@GetMapping("/{id}") new *
@Operation(
    summary = "Get category by id",
    description = "Retrieve a category by id"
)
public ResponseEntity<Category> getById(@PathVariable UUID id) {
    try {
        Category category = categoryService.getById(id);
        return ResponseEntity.ok(category);
    } catch (CategoryNotFoundByIdException e) {
        return ResponseEntity.notFound().build();
    }
}
```

Example for Not found (404) use case

We can enhance by creating a handler for exception, by throwing the exception in the controller

```
@GetMapping(🌐 "{id}")  👤 Elie Daher
@Operation(
    summary = "Get category by id",
    description = "Retrieve a category by id"
)
public ResponseEntity<Category> getById(@PathVariable UUID id) throws CategoryNotFoundByIdException {
    Category category = categoryService.getById(id);
    return ResponseEntity.ok(category);
}
```

Example for Not found (404) use case

And we will have a global exception handler class, and every exception that is thrown at the controller layer, can be handled in this class.

In the case of Category not found by id it will return a 404 response

```
@ControllerAdvice  // Elie Daher
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler({ // Elie Daher
        CategoryNotFoundByIdException.class,
        PostNotFoundByIdException.class
    })
    public ResponseEntity<String> handleNotFoundException(Exception ex) {
        logger.warn("[NOT FOUND] {}", ex.getMessage());
        return ResponseEntity
            .status(404)
            .body(ex.getMessage());
    }
}
```

Make the changes to make sure all endpoints return a http status and all exceptions are handled



Sync with Github

add http status code and handle exceptions